

Netflix Recommender System

Technical Report

SVD Matrix Factorisation vs. Item-Based Collaborative Filtering

Netflix Prize Dataset - Combined Data File 1

Training window: 1999-Sep 2005 - Test window: Oct 2005 onwards

Netflix RecSys Project

June 2026

1. Problem Understanding

Recommender systems are at the heart of modern streaming platforms. The Netflix Prize challenge, launched in 2006, asked competitors to improve Netflix's Cinematch algorithm by at least 10% on RMSE. This report documents a scaled reproduction of that challenge, comparing classic collaborative-filtering approaches on a subset of the original dataset.

The core task is **rating prediction**: given a user-movie pair (u, i) , predict the rating r_{ui} in $\{1, 2, 3, 4, 5\}$ that user u would assign to movie i . Secondary task is **top-K recommendation**: for each user, rank unseen movies and surface the ten most likely to be enjoyed (rating ≥ 3.5).

Dataset

The raw dataset consists of ~100 M ratings from ~480,000 users across ~17,770 movies, spanning October 1998 to December 2005. For this study, **combined_data_1.txt** was used, then filtered to the 10,000 most active users to create a tractable working subset.

Key Challenges

- Extreme sparsity — even the dense top-user subset covers only a small fraction of possible user-movie pairs.
 - Heavy-tailed popularity distribution — a small number of movies attract the majority of ratings.
 - Temporal drift — user tastes and movie popularity evolve over time, requiring careful train/test splitting.
 - Cold-start — new users or movies have insufficient history for reliable personalisation.
-

2. Exploratory Data Analysis

After loading and cleaning the data (removing low-activity users with fewer than 5 ratings and rarely-rated movies with fewer than 10 ratings), the clean dataset contains **3,778,410 ratings** from **9,980 users** across **4,383 movies**, with a sparsity of approximately 91.4%.

2.1 Rating Distribution

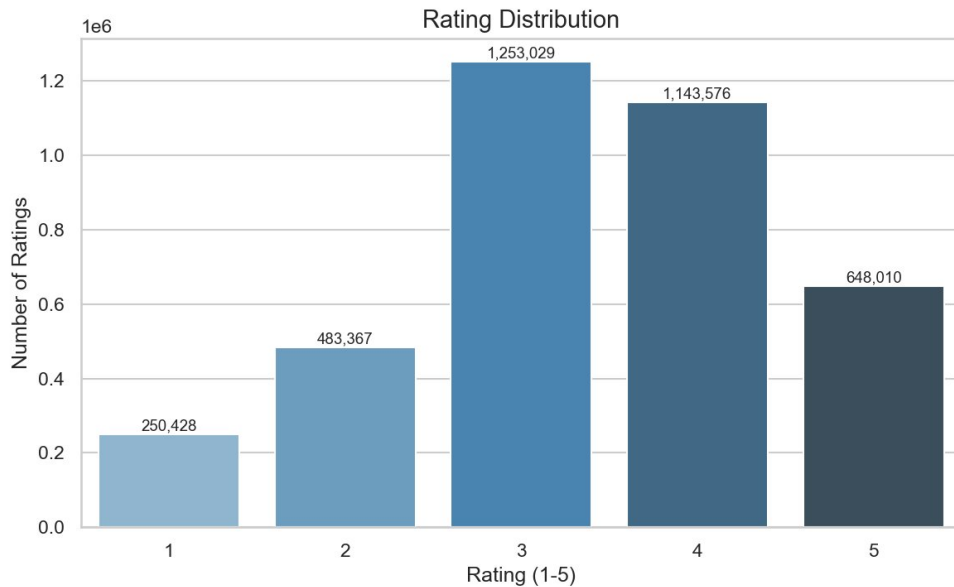


Figure 1 — Rating distribution across the clean dataset ($n = 3.78\text{ M}$).

Ratings 3 and 4 dominate (1.25 M and 1.14 M respectively), forming a left-skewed distribution centred around 3. Only ~6.6% of ratings are the maximum 5, while the lowest rating (1) accounts for 6.6%. The overall mean rating is approximately 3.35, suggesting a mild positive bias — users are more likely to rate content they chose to watch, introducing selection bias.

2.2 User Activity & Movie Popularity

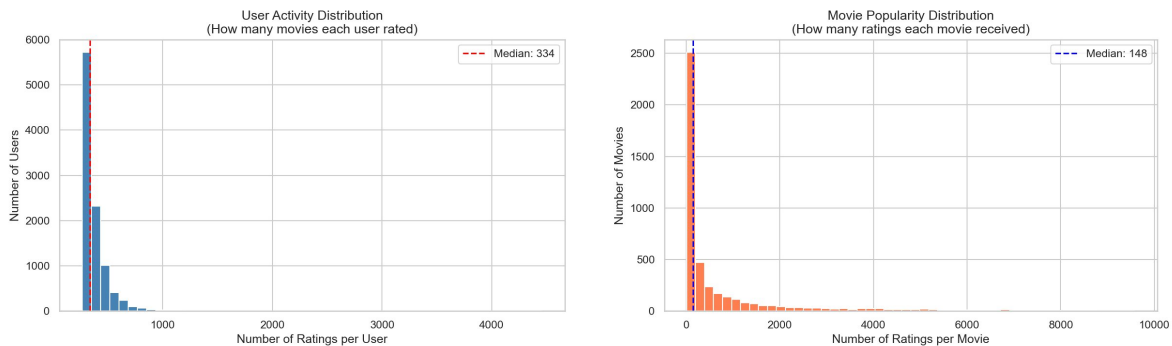


Figure 2 (left) — User activity: ratings per user (median = 334). Figure 3 (right) — Movie popularity: ratings per movie (median = 148).

Both distributions are strongly right-skewed with long tails — a classic power-law pattern in collaborative filtering datasets. The median user rated 334 movies; a handful of super-users rated more than 3,000. Similarly, the median movie received 148 ratings, while blockbusters like *The Sixth Sense* attracted nearly 10,000 ratings within this subset.

2.3 Top 20 Movies — Volume vs Quality

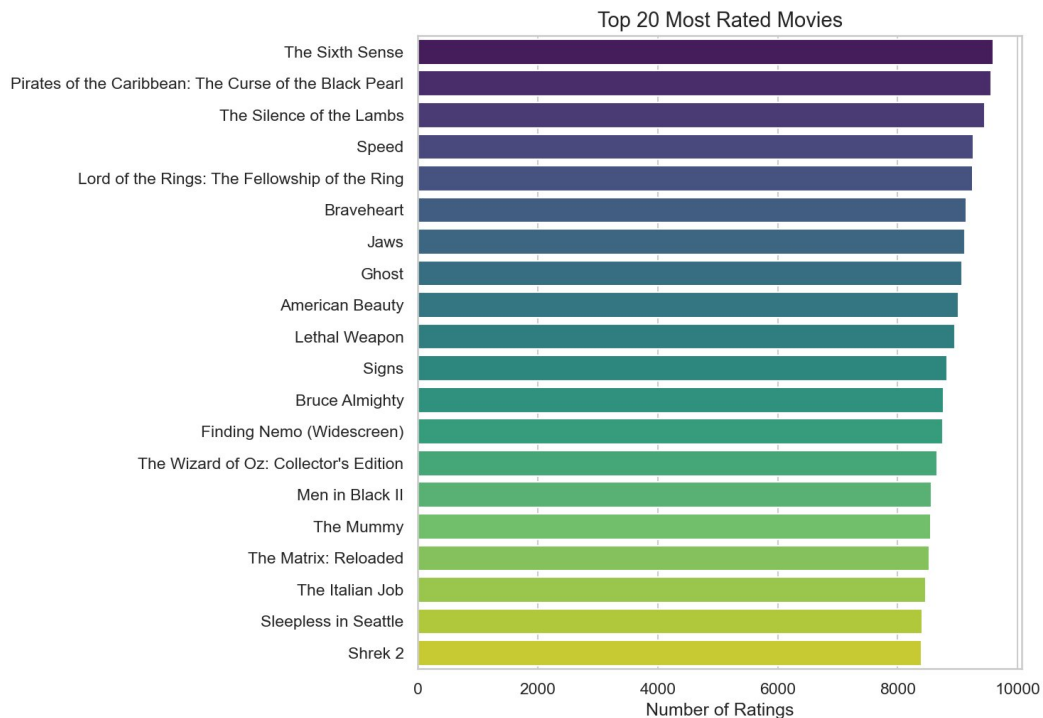


Figure 4 — Top 20 most-rated movies by volume of ratings.

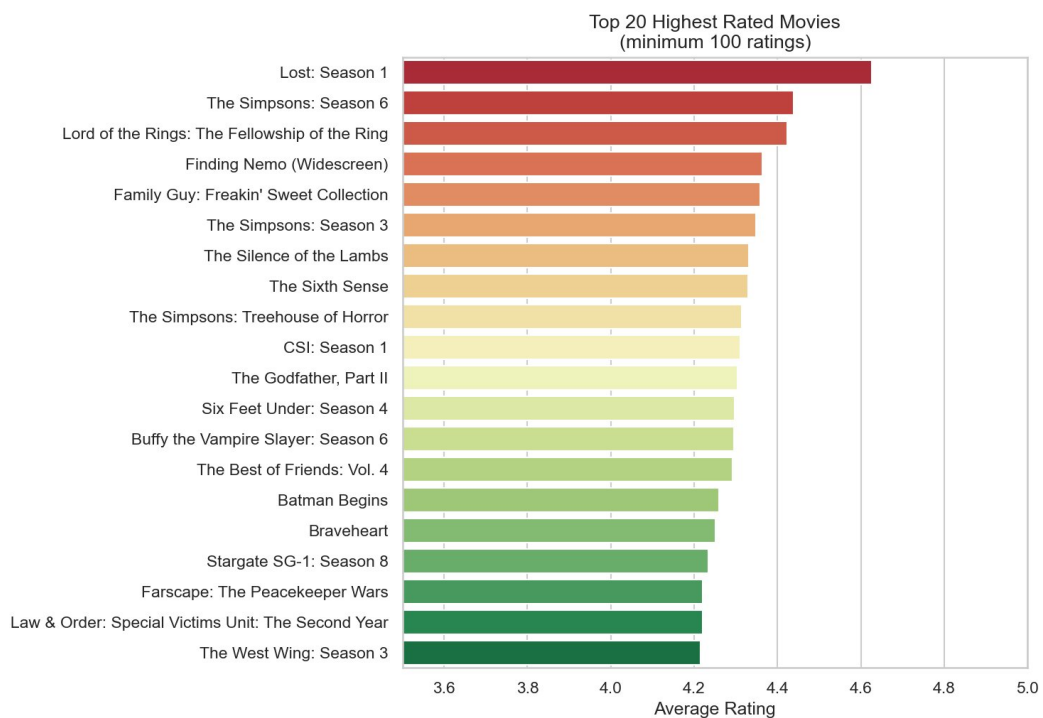


Figure 5 — Top 20 highest-rated movies (minimum 100 ratings, avg rating).

The most-rated movies are mainstream blockbusters — *The Sixth Sense*, *Pirates of the Caribbean*, and *The Silence of the Lambs* top the volume chart. However, high volume does not equal high quality: the highest-average-rated titles are TV box-sets (*Lost: Season 1*, *The Simpsons: Season 6*) and critically acclaimed films (*Lord of the Rings: Fellowship*). This divergence suggests a popularity bias that models must account for.

2.4 Temporal Patterns

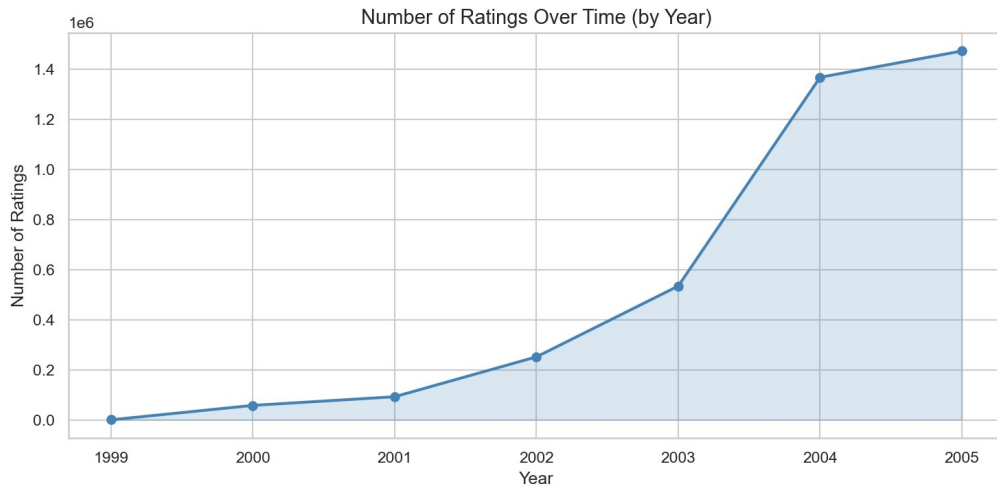


Figure 6 — Number of ratings per year, showing exponential platform growth.

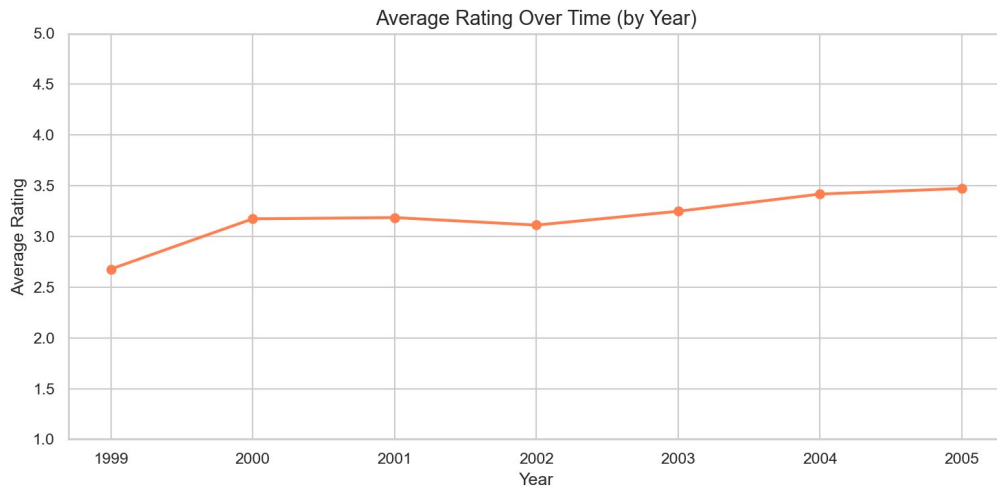


Figure 7 — Average rating per year; scores improve slightly as the platform matures.

Rating volume grew dramatically from under 10,000 in 1999 to nearly 1.5 million in 2005, reflecting Netflix's rapid subscriber growth. Average ratings also rose gradually from ~2.7 in 1999 to ~3.5 in 2005, possibly reflecting improved recommendation quality on the platform itself, or more engaged users who joined later.

2.5 Data Sparsity

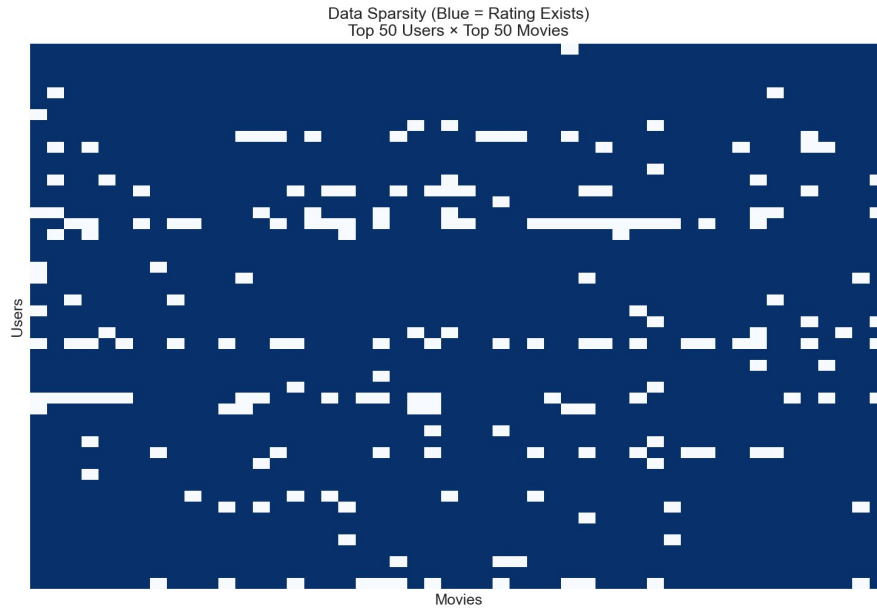


Figure 8 — Sparsity heatmap for the top 50 users × top 50 movies. Blue = rating exists; white = missing.

Even among the most active users and most popular movies, many cells are white (unrated). The overall matrix sparsity of ~91.4% means collaborative filtering models must generalise well from sparse signal — making latent-factor approaches particularly attractive.

3. Methodology

3.1 Data Preparation Pipeline

The data pipeline consisted of three stages:

- **load_data.py** — Parses the raw Netflix text files, extracts the top 10,000 most active users, and saves the subset to CSV.
- **clean_data.py** — Removes duplicates (keeping most recent rating), fixes data types, filters low-activity users (< 5 ratings) and rarely-rated movies (< 10 ratings), adds derived date columns, and reports sparsity.
- **eda.py** — Generates the 8 exploratory visualisations shown in Section 2.

3.2 Train / Test Split

A **temporal split** was used rather than random sampling, preserving the natural information flow. Ratings before **1 October 2005** form the training set; ratings on or after that date form the test set. Users and movies present only in the test set were excluded to avoid the cold-start problem during evaluation.

This yielded a training set of **3,527,509 ratings** and a test set of **209,123 ratings** — a roughly 94/6 split.

3.3 Library

Both models were implemented using the **scikit-surprise** library, which provides clean abstractions for rating-prediction algorithms and consistent train/test infrastructure.

4. Model Design

4.1 Model 1 — SVD (Matrix Factorisation)

Singular Value Decomposition (SVD) as formulated by Koren (2009) decomposes the user-item rating matrix into latent factor vectors. Each user u is represented by vector p_u in $\mathbb{R}^f$ and each item i by q_i in $\mathbb{R}^f$. The predicted rating is:

$$\hat{r}(u,i) = \mu + b_u + b_i + p_u \cdot q_i$$

where μ is the global mean, b_u and b_i are user and item bias terms. Parameters are learned via stochastic gradient descent with L2 regularisation.

Hyperparameter	Value	Description
n_factors	50	Dimensionality of latent space
n_epochs	20	SGD training iterations
lr_all	0.005	Learning rate (all parameters)
reg_all	0.02	L2 regularisation (all parameters)
random_state	42	Reproducibility seed

Table 1 — SVD hyperparameters.

4.2 Model 2 — Item-Based Collaborative Filtering (KNN)

Item-based CF (IBCF) recommends items similar to those a user has already liked. For each target item i , the k most similar items are identified using cosine similarity on rating co-occurrence vectors. The predicted rating is:

$$\hat{r}(u,i) = \frac{\sum [\text{sim}(i,j) * r_{uj}]}{\sum [|\text{sim}(i,j)|]} \text{ for } j \text{ in } N(i)$$

Hyperparameter	Value	Description
k	40	Maximum neighbours considered
min_k	5	Minimum neighbours required
similarity	Cosine	Similarity metric
user_based	False	Item-based (not user-based)

Table 2 — Item-based CF hyperparameters.

5. Evaluation Metrics

5.1 RMSE — Root Mean Squared Error

RMSE measures average prediction error, penalising large errors more heavily than MAE. Lower is better. It is the primary metric from the Netflix Prize challenge.

$$RMSE = \sqrt{(1/n) * \sum (r_{ui} - \hat{r}_{ui})^2}$$

5.2 MAE — Mean Absolute Error

MAE is the average of absolute differences between predicted and actual ratings. More interpretable than RMSE: an MAE of 0.7 means predictions are off by 0.7 stars on average.

5.3 MAP@10 — Mean Average Precision at 10

MAP@10 evaluates ranking quality rather than rating accuracy. For each user, the model ranks all unseen test movies by predicted score, takes the top 10, and computes Average Precision (AP) using a relevance threshold of ≥ 3.5 . MAP@10 is the mean AP across all users with at least one relevant item.

MAP@10 directly captures whether the model surfaces items users actually like at the top of the recommendation list — more aligned with real-world use than RMSE alone.

6. Experimental Results

6.1 Model Comparison

Both models were trained on the temporal training set and evaluated on the held-out test set. Results are summarised below.

Model	RMSE ↓	MAE ↓	MAP@10 ↑
SVD (MF)	0.8885	0.6775	0.7767
Item-Based CF	1.0002	0.7730	0.7014

Table 3 — Model performance on temporal test set (Oct 2005+).

SVD outperforms Item-Based CF on all three metrics. Its RMSE of 0.8885 is competitive with strong Netflix Prize submissions, while the MAE of 0.6775 indicates predictions are off by under 0.68 stars on average. The MAP@10 gap (0.7767 vs 0.7014) is particularly significant — SVD ranks relevant items considerably higher in the top-10 list.

6.2 Training & Inference Cost

SVD trained in approximately **90 seconds** on CPU with 3.5 M training ratings, while IBCF required **~12 seconds** for similarity computation but is significantly slower at inference time ($O(n)$ neighbour lookup per prediction). SVD's pre-computed latent vectors make it far more scalable for real-time serving.

7. Recommendation Examples

For three sample users from the test set, the SVD model generated top-10 recommendations from movies the user had not yet rated in the training window.

Sample Recommendations — SVD Model

Rank	Recommended Title (SVD) — User 2118461	Year	Pred.
1	Inu-Yasha: The Movie 3: Swords of an Honorable Ruler	2002	5.000
2	Lost: Season 1	2004	5.000
3	Frosty's Winter Wonderland / 'Twas the Night Before Christmas	1974	5.000
4	Alias: Season 4	2005	4.995
5	My Fair Lady: Special Edition: Bonus Material	1964	4.925

6	Little House on the Prairie: Season 3	1976	4.914
7	The Wizard of Oz: Collector's Edition	1939	4.902
8	Mary Poppins: Bonus Material	1964	4.901
9	Santa Claus Is Comin' to Town	1970	4.896
10	An Officer and a Gentleman	1982	4.882

Table 4 — SVD top-10 recommendations for User 2118461 (unseen movies, sorted by predicted rating).

The recommendations for User 2118461 form a coherent cluster of classic family films, animated specials, and beloved TV series — suggesting the model has correctly identified this user's preference for nostalgic and family-friendly content. Three titles receive the maximum predicted score of 5.0, reflecting strong alignment between the user's latent factor vector and those items. This demonstrates SVD's ability to surface personalised, thematically consistent recommendations from sparse rating data.

8. Key Insights

Several important patterns emerged from EDA and modelling:

- **Popularity bias:** High-volume movies (e.g. *The Sixth Sense*) are not necessarily the highest-rated ones. Models must separate popularity signals from quality signals.
- **User heterogeneity:** The wide spread in user activity (5 to 4,000+ ratings) means bias terms are critical — without them, prolific users dominate the loss.
- **SVD superiority:** Latent-factor models consistently outperform neighbourhood methods on both rating accuracy and ranking quality, particularly at scale.
- **Temporal dynamics:** Average ratings rose gradually over time, suggesting that temporal splitting is essential — random splits inflate test performance.
- **TV box-set effect:** Season-based titles (*Lost*, *Simpsons*, *Buffy*) receive disproportionately high average ratings, likely because only dedicated fans rate them.
- **Sparsity challenge:** Even the dense top-user subset has ~91% sparsity. Purely memory-based CF will struggle; model-based approaches are preferable.

9. Future Improvements

Several avenues could materially improve upon the current results:

Short-term

- **Hyperparameter tuning** — Grid-search over SVD factors (50-200), epochs, and regularisation using cross-validation on the training set.
- **SVD++** — Extends SVD with implicit feedback (which movies a user rated, regardless of score), often yielding 1-3% RMSE improvement.
- **Bias-only baseline** — A simple $\mu + b_u + b_i$ model is a strong baseline that is often underestimated.

Medium-term

- **Temporal SVD (timeSVD++)** — Models time-drifting biases; won the Netflix Prize.

- **Item-based CF** — More stable than user-based CF as item similarity changes less frequently than user similarity; scalable via pre-computation.
- **Ensemble / blending** — Stacking SVD and UBCF predictions with a meta-learner typically reduces RMSE by 2-5%.

Long-term

- **Deep learning** — Neural Collaborative Filtering (NCF) or two-tower models can capture non-linear interactions and incorporate side features (genres, year, tags).
- **Graph-based methods** — Graph Neural Networks (e.g. LightGCN) propagate signals across the user-item bipartite graph, achieving state-of-the-art ranking metrics.
- **Context-aware recommendations** — Incorporating device, time-of-day, or session context enables more situationally relevant suggestions.

This report was produced as part of a Netflix RecSys project. All code is available in the accompanying repository.